Name : JETHAVA JAY VINODBHAI

Enrollment No. : 250283152004

1. Demonstration of main Pandas methods

a. Import necessary libraries (numpy, pandas and matplotlib).

```
import pandas as pd
import numpy as np
import matplotlib
```

b. Read the data using read_csv function of pandas and store into a dataframe variable.

```
df = pd.read_csv('/content/Lab-2-Data-Telecom_Churn.csv')
```

c. Use head, shape, columns, info and describe functions to get details about the data.

```
df.head()
```

	state	account length	area code	phone number	international plan	voice mail plan	number vmail messages	total day minutes	tota da call
0	KS	128	415	382-4657	no	yes	25	265.1	11
1	OH	107	415	371-7191	no	yes	26	161.6	12
2	NJ	137	415	358-1921	no	no	0	243.4	11
3	OH	84	408	375-9999	yes	no	0	299.4	7
4	OK	75	415	330-6626	yes	no	0	166.7	11

5 rows × 21 columns

```
df.shape
```

```
(3333, 21)
```

```
df.columns
```

```
Index(['state', 'account length', 'area code', 'phone number',
       'international plan', 'voice mail plan', 'number vmail messages',
       'total day minutes', 'total day calls', 'total day charge',
       'total eve minutes', 'total eve calls', 'total eve charge',
       'total night minutes', 'total night calls', 'total night charge',
       'total intl minutes', 'total intl calls', 'total intl charge',
       'customer service calls', 'churn'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3333 entries, 0 to 3332
Data columns (total 21 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   state                   3333 non-null   object 
 1   account length          3333 non-null   int64  
 2   area code               3333 non-null   int64  
 3   phone number            3333 non-null   object 
 4   international plan      3333 non-null   object 
 5   voice mail plan         3333 non-null   object 
 6   number vmail messages   3333 non-null   int64  
 7   total day minutes       3333 non-null   float64
 8   total day calls         3333 non-null   int64  
 9   total day charge        3333 non-null   float64
 10  total eve minutes       3333 non-null   float64
 11  total eve calls         3333 non-null   int64  
 12  total eve charge        3333 non-null   float64
 13  total night minutes     3333 non-null   float64
 14  total night calls       3333 non-null   int64  
 15  total night charge      3333 non-null   float64
 16  total intl minutes      3333 non-null   float64
 17  total intl calls        3333 non-null   int64  
 18  total intl charge       3333 non-null   float64
 19  customer service calls  3333 non-null   int64  
 20  churn                   3333 non-null   bool   
dtypes: bool(1), float64(8), int64(8), object(4)
memory usage: 524.2+ KB
```

```
df.describe()
```

	account length	area code	number vmail messages	total day minutes	total day calls	total ch
count	3333.000000	3333.000000	3333.000000	3333.000000	3333.000000	3333.000
mean	101.064806	437.182418	8.099010	179.775098	100.435644	30.56
std	39.822106	42.371290	13.688365	54.467389	20.069084	9.25
min	1.000000	408.000000	0.000000	0.000000	0.000000	0.00
25%	74.000000	408.000000	0.000000	143.700000	87.000000	24.43
50%	101.000000	415.000000	0.000000	179.400000	101.000000	30.50
75%	127.000000	510.000000	20.000000	216.400000	114.000000	36.79
max	243.000000	510.000000	51.000000	350.800000	165.000000	59.64

d. Convert column ‘Churn’ datatype from bool to int64 using astype function.

```
df['churn'] = df['churn'].astype('int64')
```

f. Use value_counts function for Churn column.

```
print(df['churn'].value_counts())
```

```
churn
0    2850
1     483
Name: count, dtype: int64
```

g. Sorting: sort by Total day charge (use ascending=False to sort in descending order) using sort_values.

```
df.sort_values(by='total day charge',ascending=False).head()
```

	state	account length	area code	phone number	international plan	voice mail plan	number vmail messages	total day minutes	t
365	CO	154	415	343-5709	no	no	0	350.8	
985	NY	64	415	345-9140	yes	no	0	346.8	
2594	OH	115	510	348-1163	yes	no	0	345.3	
156	OH	83	415	370-9116	no	no	0	337.4	
605	MO	112	415	373-2053	no	no	0	335.5	

5 rows × 21 columns

2 Indexing and Retireiving :

a. What is the proportion of churned users in our dataframe? (Use mean())

```
print("Proportion of Churn Users : ")
print(df["churn"].mean())
```

```
Proportion of Churn Users :
0.14491449144914492
```

b. What are average values of numerical features for churned users?

```
print(df[df["churn"]==1].mean(numeric_only=True))
```

```
account length            102.664596
area code                 437.817805
number vmail messages       5.115942
total day minutes         206.914079
total day calls           101.335404
total day charge           35.175921
total eve minutes         212.410145
total eve calls           100.561077
total eve charge           18.054969
total night minutes       205.231677
total night calls         100.399586
total night charge          9.235528
total intl minutes         10.700000
total intl calls            4.163561
total intl charge           2.889545
customer service calls      2.229814
```

```
churn                         1.000000
dtype: float64
```

c. How much time (on average) do churned users spend on the phone during daytime?

```
print(df[df["churn"]==1]['total day minutes'].mean())
```

```
206.91407867494823
```

d. What is the maximum length of international calls among loyal users (Churn == 0) who do not have an international plan?

```
print(df[(df['churn']==0) & (df['international plan']=='No')]['total intl ca
```

```
nan
```

3. Applying functions to cells, columns and rows:

a. Change International Plan column values form Yes and No to True and False respectively using map function.

```
df["international plan"] = df["international plan"].map({'Yes':True,'No':Fal
```

b. Change Voice mail Plan column values form Yes and No to True and False respectively using map function.

```
df["voice mail plan"] = df["voice mail plan"].map({'Yes':True,'No':False})
```

Grouping

b) Describe "Total day minutes", "Total eve minutes", "Total night minutes" grouped by Churn

```
df.groupby("churn")[[
    "total day minutes",
    "total eve minutes",
    "total night minutes"
]].describe()
```

	total day minutes								total e
	count	mean	std	min	25%	50%	75%	max	count
churn									
0	2850.0	175.175754	50.181655	0.0	142.825	177.2	210.30	315.6	2850.0
1	483.0	206.914079	68.997792	0.0	153.250	217.6	265.95	350.8	483.0

2 rows × 24 columns

Summary Tables

a & b) Crosstab for Churn

```
pd.crosstab(df["churn"], columns="count")
```

col_0	count
churn	
0	2850
1	483

```
pd.crosstab(index=df["churn"], columns="count")
```

col_0	count
churn	
0	2850
1	483

c) Crosstab for Churn and International Plan

```
pd.crosstab(df["churn"], df["international plan"])
```

international plan	
churn	

d) Crosstab with percentages

```
pd.crosstab(
    df["churn"],
    df["international plan"],
    normalize=True
)
```

international plan	churn

```
# Row-wise Percentages :
pd.crosstab(
    df["churn"],
    df["international plan"],
    normalize="index"
)
```

international plan	churn

```
# Column-wise percentages :
pd.crosstab(
    df["churn"],
    df["international plan"],
    normalize="columns"
)
```

international plan	churn

f & g) Pivot Table

```
df.pivot_table(
    values=[
        "total day calls",
        "total eve calls",
        "total night calls"
    ],
    index="area code",
    aggfunc="mean"
)
```

area code	total day calls	total eve calls	total night calls
408	100.496420	99.788783	99.039379
415	100.576435	100.503927	100.398187
510	100.097619	99.671429	100.601190

6. DataFrame Transformations

a) Total calls using insert()

```
total_calls = (
    df["total day calls"] +
    df["total eve calls"] +
    df["total night calls"] +
    df["total intl calls"]
)

df.insert(len(df.columns), "Total Calls", total_calls)
```

b) Total charge without insert()

```
df["Total Charge"] = (
    df["total day charge"] +
    df["total eve charge"] +
    df["total night charge"] +
    df["total intl charge"]
)
```

7. Analysis involving more than one column

a) Churn vs International Plan

```
pd.crosstab(df["churn"], df["international plan"])
```

international plan	churn

```
# With percentages:
pd.crosstab(
    df["churn"],
```

```
    df["international plan"],
    normalize="index"
)
```

international plan	churn

b) Churn vs Customer Service Calls

```
pd.crosstab(
    df["churn"],
    df["customer service calls"]
)
```

customer service calls	0	1	2	3	4	5	6	7	8	9
churn										
0	605	1059	672	385	90	26	8	4	1	0
1	92	122	87	44	76	40	14	5	1	2

c) Churn vs Customer Service Calls > 3

```
#Create a new Boolean column:
df["customer service calls >3"] = df["customer service calls"] > 3
```

```
#Now create crosstab:
pd.crosstab(
    df["churn"],
    df["customer service calls >3"]
)
```

customer service calls >3	False	True
churn		
0	2721	129
1	345	138

```
#With percentages:
pd.crosstab(
    df["churn"],
    df["customer service calls >3"],
    normalize="index"
)
```

customer service calls >3	False	True
churn		
0	0.954737	0.045263
1	0.714286	0.285714

d) Churn with both International Plan and Customer Service Calls >3

```
pd.crosstab(
    df["churn"],
    [
        df["international plan"],
```